

Hydra — Network Login Brute-Forcing

Hydra sits at a different point in the attack chain than everything you've covered so far — recon and enumeration tell you *what services are running and what usernames might exist*; Hydra is where you actually try to **walk through the front door** by throwing credentials at those services until one works. It's the tool that turns a list of usernames from your enumeration guides and a wordlist into an actual authenticated session, which is why it sits right at the boundary between reconnaissance and initial access. Worth locking in early: Hydra doesn't find vulnerabilities — it exploits the *absence* of one (rate limiting, lockout policy, MFA) against a login mechanism you've already identified through Nmap/enumeration work from earlier guides.

1. What Hydra Actually Does — The Foundation

Hydra ("THC-Hydra") is a parallelized network login cracker that supports dozens of protocols. At its core, it's a loop:

1. Take a TARGET (host + service/port)
2. Take a CREDENTIAL SOURCE (username list, password list, or both)
3. Take a SERVICE MODULE that knows how to speak that protocol's login handshake (ssh, ftp, http-post-form, smb, rdp, mysql, etc.)
4. Attempt login after login, combining usernames x passwords, IN PARALLEL across multiple threads
5. Report back which combination(s), if any, succeeded

The module is the entire ballgame. Hydra itself doesn't "understand" SSH or a web login form — each protocol has a dedicated module that knows how to construct a valid login attempt and how to distinguish success from failure for that specific service. This is why most real-world Hydra work isn't about the brute-force logic (that part is trivial) — it's about correctly configuring the module for the target's specific login behavior, especially for HTTP forms, which is where nearly all practical difficulty lives (Section 6).

Attack surface Hydra targets → any network-facing AUTHENTICATION mechanism: SSH, FTP, Telnet, SMB, RDP, VNC, MySQL/MSSQL/PostgreSQL, HTTP basic auth, HTTP form-based login, POP3/IMAP/SMTP, and more

Attack surface Hydra does NOT target → application logic flaws, injection points, anything that isn't a login prompt

PART 1: Core Mechanics & Syntax

2. Basic Command Structure

```
hydra [options] -l <username> -P <wordlist> <target> <service>
```

Breaking down the pieces you'll use in nearly every invocation:

-l <user>	single username
-L <userlist>	file of usernames to try
-p <pass>	single password
-P <passlist>	file of passwords to try
-C <file>	combined user:pass file (skips -l/-L and -p/-P entirely)
-t <n>	number of parallel tasks/threads (default 16)
-s <port>	non-default port
-V / -v	verbose – show every attempt as it happens
-f	stop as soon as ONE valid pair is found (single target)
-F	stop as soon as ONE valid pair is found (across ALL targets)
-o <file>	write found credentials to a file

Basic SSH example — directly extending your active recon guide's Nmap findings:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10
```

Username list + password list combined (full combinatorial attack):

```
hydra -L users.txt -P passwords.txt ssh://192.168.1.10 -t 4
```

3. Single Credential Pair vs Full Wordlist Attack

Two fundamentally different use cases that get conflated:

CREDENTIAL STUFFING mode → you have a KNOWN username (from enumeration, OSINT, a breach dump) and are testing it against a large password list – this is -l + -P

PURE BRUTE FORCE mode → you don't know either value with confidence, so you're testing large lists against each other – this is -L + -P, and the attempt count multiplies (users × passwords), making it far noisier and slower

In real engagements, credential stuffing mode is overwhelmingly more common and realistic — pure brute force across both axes is loud, slow, and usually the first thing account lockout policies are designed to catch.

4. Thread Tuning — Speed vs Stability vs Detection

The `-t` flag controls concurrency, and getting it wrong breaks things in different directions depending on which way you err:

T00 HIGH (-t 64+) → can crash fragile services (especially FTP/RDP), trigger IDS/IPS and account lockout policies almost immediately, and produce unreliable false negatives on slow services

T00 LOW (-t 1-2) → attack takes drastically longer, but is far less likely to trigger lockout thresholds or get flagged by rate-based detection

Reasonable defaults by protocol:

SSH → -t 4 (SSH is expensive per-connection; high thread counts often just cause failures, not speed)

HTTP forms → -t 16-40 (usually stateless and tolerant of concurrency)

SMB/RDP → -t 1-4 (these protocols are notorious for locking accounts or crashing under concurrent auth attempts)

5. Session Resuming

Long-running attacks against large wordlists don't need to restart from zero if interrupted:

```
hydra -l admin -P big_wordlist.txt ssh://192.168.1.10 -o results.txt
# ...interrupted (Ctrl+C)...
hydra -R
```

- `R` restores and resumes the most recently interrupted session using Hydra's own session state file — useful on long engagements where a wordlist run spans hours.

PART 2: Protocol-Specific Attacks & Advanced Usage

6. HTTP Form-Based Login — The Hardest and Most Common Real-World Case

Unlike SSH/FTP where success/failure is protocol-native, HTTP login forms require **telling Hydra how to recognize failure**, since a failed login is still an HTTP 200 response — it just contains different content. This is the module that trips people up the most, and directly builds on your earlier web security guides' handling of form submission and cookies/sessions.

```
hydra -l admin -P passwords.txt <target> http-post-form \
  "/login:username=^USER^&password=^PASS^:F=Invalid username or password"
```

Breaking down the module string, which is genuinely the crux of this whole section:

/login	→ the path the form POSTs to
username=^USER^	→ the form field name, with ^USER^ as the placeholder Hydra substitutes each username attempt into
password=^PASS^	→ same pattern for the password field
F=<string>	→ FAILURE condition — a string that only appears on a FAILED login (e.g. an error message). Hydra treats ANY response containing this string as a failed attempt, and anything else as a potential success
S=<string>	→ alternative: SUCCESS condition — a string that only appears after a successful login (e.g. "Logout" or "Welcome"), often more reliable than F= since failure pages can vary in subtle ways

Handling CSRF tokens / session cookies in the form: many login forms require a token scraped from the initial GET request before the POST will even be accepted — Hydra can't do this natively for dynamic tokens, which is a genuine limitation. Common workarounds:

- Check whether the token is actually validated server-side at all (many apps generate a token but don't check it — test manually first)
- Use a cookie captured from a single valid session if the token isn't actually per-request
- Fall back to Burp Intruder for form logins with real per-request CSRF validation — Hydra is the wrong tool once dynamic tokens are strictly enforced

7. FTP, Telnet, SMB — Straightforward Protocol Modules

These protocols have native, unambiguous success/failure signaling, making them far simpler than HTTP forms:

```
hydra -l admin -P passwords.txt ftp://192.168.1.10
hydra -L users.txt -P passwords.txt telnet://192.168.1.10
hydra -l administrator -P passwords.txt smb://192.168.1.10
```

SMB in particular connects directly back to your network scanning guide's enumeration work — a username list pulled from SMB null-session enumeration feeds straight into an SMB Hydra run against the same host.

8. RDP and Database Services

```
hydra -l administrator -P passwords.txt rdp://192.168.1.10
hydra -l root -P passwords.txt mysql://192.168.1.10
hydra -l sa -P passwords.txt mssql://192.168.1.10
```

Important caveat specific to RDP: RDP is exceptionally prone to account lockout and can lock out legitimate domain accounts after only a handful of failed attempts under default Active Directory policy — this is one of the few Hydra targets where you should assume real operational risk to the client environment, not just detection risk, before running anything beyond a handful of carefully chosen attempts.

9. Using a Combined Credential File

When you have a curated list of *known* username:password pairs (e.g. from a breach dump, default credential list, or prior recon) rather than two independent lists:

```
hydra -C creds.txt ssh://192.168.1.10
```

Where `creds.txt` is formatted `username:password` per line. This is a **pairing**, not a combinatorial run — each line is tried as-is, not multiplied against every other line, which makes it far faster and far quieter than `-L + -P` mode.

10. Avoiding False Positives

Hydra can report false successes, particularly on custom or unusual services:

- Manually verify ANY reported "valid" credential pair by logging in through a normal client, not just trusting Hydra's output
- For HTTP forms, false positives are the single most common issue – double-check your F= or S= string actually appears reliably and ONLY on the condition you think it does, ideally by testing one known-bad and one known-good login manually first
- Watch for services that return a generic 200/OK regardless of credentials (common on some legacy or misconfigured apps) – Hydra will report every attempt as a success in this case

11. Real-World Impact

Initial access via weak/default credentials → the single most common real-world outcome; default vendor credentials on exposed management interfaces remain extremely common

Credential reuse exposure → a password found valid on one service is routinely tried against every other exposed service on the same host/network (directly feeds into lateral movement)

Account lockout / denial of service → an unintended but real consequence of careless thread/attempt tuning against production accounts

12. Detection & Defensive Testing Methodology

If you're testing a target's resilience to this class of attack (rather than just running the tool), the methodology is:

1. Identify the account lockout threshold (if any) – how many failed attempts trigger a lockout, and for how long
2. Run a deliberately small, controlled attempt count (well under the lockout threshold) to confirm the login mechanism's actual failure/success signaling matches what you expect
3. Check whether failed attempts are logged/alerted at all - many organizations have NO monitoring on authentication failure volume, which is itself a finding worth reporting
4. Test whether MFA is actually enforced on EVERY path to the service, not just the primary login page (a common gap: MFA on the web UI but not on an API endpoint or legacy protocol using the same backend credentials)
5. Confirm whether rate limiting is IP-based or account-based - IP-based limiting is o

ften trivially bypassed via a rotating proxy/VPN, which is worth calling out explicitly in a report

13. Prevention

- Enforce account lockout after a small number of failed attempts (industry-standard guidance: 5-10 attempts, though this must be balanced against denial-of-service risk from an attacker deliberately locking out legitimate accounts)
- Enforce MFA on every authentication path to a service, not just the primary web login – legacy protocols (FTP, Telnet, older RDP/SMB configurations) frequently lack any MFA hook at all and should be disabled or tightly restricted where MFA can't be enforced
- Rate-limit authentication attempts server-side, ideally combining both per-account AND per-source-IP limiting so neither axis alone can be trivially bypassed
- Eliminate default/vendor credentials on every exposed service as a baseline hardening step - this remains one of the highest-yield findings in real engagements
- Use CAPTCHA or equivalent challenge-response on public-facing login forms after a small number of failed attempts, to disrupt automated tooling specifically
- Monitor and alert on authentication failure volume/velocity per account and per source - a sudden spike is one of the highest-signal indicators of an active brute-force attempt in progress
- Disable legacy/plaintext protocols (Telnet, unencrypted FTP) entirely where not operationally required, removing the attack surface rather than just hardening it

14. Practical Lab Example

Hydra doesn't have a dedicated PortSwigger lab track (it's a network tool, not a web-app vulnerability class), but it fits naturally into the following practice environments:

1. Metasploitable 2 - directly reuse the host you've already been scanning in your network scanning guide. Run Hydra against its FTP, SSH, and Telnet services using known default credentials as a first pass, then a real wordlist run once you've confirmed the module syntax works
2. DVWA's "Brute Force" module (low/medium/high difficulty) -

the best environment for practicing the http-post-form module specifically (Section 6), since DVWA's low setting has no CSRF token, medium adds basic delay-based defenses, and high adds a CSRF token - letting you feel the exact limitation described in Section 6 firsthand

3. TryHackMe rooms tagged "brute force" or "hydra" - several walk through the http-post-form syntax step by step against a deliberately tricky custom login form, which is the single most valuable rep to get before using Hydra on a real engagement

Suggested practice order:

1. SSH/FTP against Metasploitable 2 first - confirms your basic syntax and thread-tuning instincts before adding HTTP complexity
2. DVWA brute force, low setting - the http-post-form module, no CSRF friction
3. DVWA brute force, high setting - forces you to confront the CSRF token limitation from Section 6 directly

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-307 (Improper Restriction of Excessive Authentication Attempts), CWE-521 (Weak Password Requirements) — the two root causes that make Hydra effective at all
OWASP Top 10:2025	A07:2025 Identification and Authentication Failures — the category directly covering missing lockout, weak credentials, and missing MFA that Hydra exploits
CVSS	Typically Medium-High depending on the exposed service; scales toward Critical when the compromised account has administrative/domain privileges, since credential access is frequently the pivot point into full compromise
Cyber Kill Chain	Exploitation (the successful login itself) → Installation/Actions on Objectives once the attacker has an authenticated foothold
MITRE ATT&CK	T1110 (Brute Force) as the parent technique, with sub-techniques T1110.001 (Password Guessing), T1110.002 (Password Cracking — less applicable here, more relevant to offline hash attacks), T1110.003 (Password Spraying — the low-and-slow, single-password-many-users variant), and T1110.004 (Credential Stuffing — Section 3's known-username mode)

Quick Reference Cheat Sheet

SYNTAX SKELETON:

```
hydra -l/-L <user(s)> -p/-P <pass(es)> -t <threads> <target>  
> <service>
```

SSH:

```
hydra -l admin -P rockyou.txt ssh://<target>
```

FTP:

```
hydra -l admin -P rockyou.txt ftp://<target>
```

SMB:

```
hydra -l administrator -P rockyou.txt smb://<target>
```

RDP (use sparingly - lockout risk):

```
hydra -l administrator -P rockyou.txt rdp://<target> -t 2
```

HTTP POST FORM:

```
hydra -l admin -P rockyou.txt <target> http-post-form \  
  "/login:username=^USER^&password=^PASS^:F=Invalid usernam  
e"
```

COMBINED CREDENTIALS (username:password pairs, no multiplication):

```
hydra -C creds.txt ssh://<target>
```

STOP ON FIRST HIT / RESUME:

- f (stop on first success, single target)
- R (resume last interrupted session)

THREAD GUIDANCE:

```
SSH: -t 4 | HTTP: -t 16-40 | SMB/RDP: -t 1-4
```